PROJECT 2

jiya02arora@gmail.com

TASK 2 – DEEP LEARNING SENTIMENT ANALYSIS USING TENSORFLOW

1. Objective

The objective of this project is to build a deep learning model capable of classifying movie reviews as positive or negative based on their textual content. Sentiment analysis is a Natural Language Processing (NLP) task widely used to understand customer opinions, feedback, and emotions.

---

2. Dataset Information

Dataset Used: IMDb Movie Reviews Dataset

Dataset Characteristics:

- 50,000 movie reviews
- Binary sentiment classification
- 25,000 training samples
- 25,000 testing samples

Target Classes:

- Positive Review
- Negative Review

Dataset Source:
TensorFlow/Keras Built-in Dataset

---

3. Technologies Used

- Python
- Google Colab
- TensorFlow
- Keras
- NumPy
- Matplotlib
- Scikit-Learn

---

4. Data Preprocessing

The following preprocessing steps were performed:

1. Loaded IMDb dataset using TensorFlow.
2. Limited vocabulary size to the most frequent words.
3. Tokenized movie reviews into numerical sequences.
4. Applied sequence padding to ensure uniform input length.
5. Created training, validation, and testing datasets.
6. Prepared data for deep learning model training.

---

5. Deep Learning Model Architecture

The sentiment analysis model was developed using TensorFlow/Keras.

Model Components:

Embedding Layer

Converts words into dense vector representations.

Spatial Dropout Layer

Reduces overfitting by randomly dropping features.

Bidirectional LSTM Layers

Captures contextual information from both forward and backward directions.

Dense Layers

Extracts higher-level features from text representations.

Output Layer

Sigmoid activation for binary sentiment classification.

---

6. Training Configuration

Training Parameters:

- Vocabulary Size: 10,000 words
- Maximum Sequence Length: 256
- Embedding Dimension: 128
- Batch Size: 64
- Epochs: 20
- Optimizer: Adam
- Loss Function: Binary Crossentropy

Callbacks Used:

- EarlyStopping
- ModelCheckpoint
- ReduceLROnPlateau

These callbacks improve model performance and prevent overfitting.

---

7. Model Training

The model was trained on the IMDb training dataset and validated using a separate validation set.

Training metrics monitored:

- Training Accuracy
- Validation Accuracy
- Training Loss
- Validation Loss
- Validation AUC

---

8. Model Evaluation

The trained model was evaluated on the test dataset using:

Evaluation Metrics

- Accuracy
- Loss
- ROC-AUC Score
- Precision
- Recall
- F1 Score

Additional Analysis

- Classification Report
- Confusion Matrix
- ROC Curve

---

9. Visualizations

The following visualizations were generated:

1. Dataset Analysis
2. Review Length Distribution
3. Training Accuracy Curve
4. Validation Accuracy Curve
5. Training Loss Curve
6. Validation Loss Curve
7. ROC Curve
8. Confusion Matrix
9. Sample Inference Results

---

10. Model Saving and Deployment Assets

The following artifacts were generated:

- imdb_sentiment_model.keras
- imdb_word_index.json
- model_config.json

These files can be used to reload the model and perform sentiment prediction on new reviews.

---

11. Sample Inference

The trained model was tested on custom movie reviews.

Example Outputs:

Review:
"This movie was absolutely fantastic and engaging."

Prediction:
Positive Sentiment

Review:
"The plot was boring and the acting was terrible."

Prediction:
Negative Sentiment

---

12. Results

Key Observations:

- The model successfully learned sentiment patterns from movie reviews.
- Bidirectional LSTM architecture improved contextual understanding.
- Validation metrics indicated good generalization performance.
- Early stopping prevented overfitting.
- The model achieved strong sentiment classification accuracy.

---

13. Applications

Potential applications of sentiment analysis include:

- Customer feedback analysis
- Product review classification
- Social media monitoring
- Brand reputation analysis

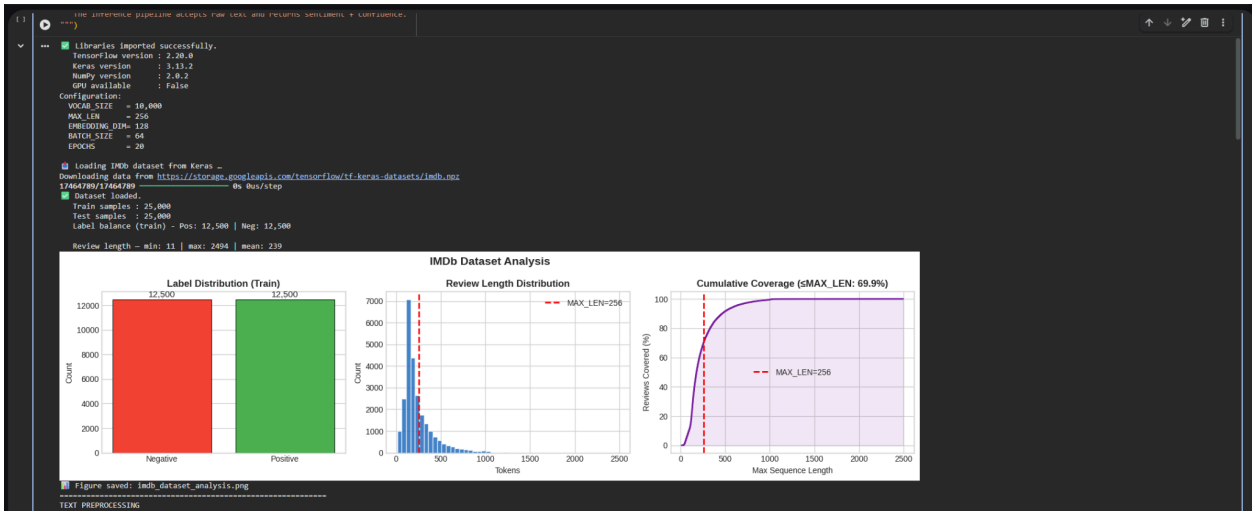
- Opinion mining
- Recommendation systems

---

14. Conclusion

A deep learning-based sentiment analysis system was successfully developed using TensorFlow and the IMDb Movie Reviews Dataset. The Bidirectional LSTM model demonstrated strong performance in classifying positive and negative reviews. The project showcases the application of Natural Language Processing and Deep Learning techniques for text classification tasks.

16.Output Screenshots



```
    The inference pipeline accepts raw text and returns sentiment + confidence.
    """)

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json
1641221/1641221 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Sample review (first 20 tokens decoded):
<START> this film was just brilliant casting location scenery story direction everyone's really suited the part they played and you

After padding:
  X_train shape : (25000, 256)
  X_test  shape : (25000, 256)

Final split:
  Train  : 21,250 samples
  Val    : 3,750  samples
  Test   : 25,000 samples
============================================================
MODEL ARCHITECTURE
============================================================
Model: "IMDb_Sentiment_Classifier"
```

Layer (type)	Output Shape	Param #
input_tokens (InputLayer)	(None, 256)	0
embedding (Embedding)	(None, 256, 128)	1,280,000
spatial_dropout (SpatialDropout1D)	(None, 256, 128)	0
bilstm_1 (Bidirectional)	(None, 256, 128)	98,816
bilstm_2 (Bidirectional)	(None, 64)	41,216
dense_1 (Dense)	(None, 64)	4,160
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
dropout_2 (Dropout)	(None, 32)	0
output (Dense)	(None, 1)	33

```
Total params: 1,426,305 (5.44 MB)
Trainable params: 1,426,305 (5.44 MB)
Non-trainable params: 0 (0.00 B)

Total parameters     : 1,426,305
Trainable parameters : 1,426,305
✅ Callbacks configured:
   • EarlyStopping   (patience=3, monitor=val_loss)
   • ModelCheckpoint (best val_auc saved)
   • ReduceLROnPlateau (factor=0.5, patience=2)
============================================================
TRAINING
============================================================
Training on 21,250 samples, validating on 3,750 samples …

Epoch 1/20
```

```
    The inference pipeline accepts raw text and returns sentiment + confidence.
    """)
============================================================
TRAINING
============================================================
Training on 21,250 samples, validating on 3,750 samples …

Epoch 1/20
333/333 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.6231 - auc: 0.6789 - loss: 0.6237
Epoch 1: val_auc improved from None to 0.89514, saving model to imdb_sentiment_model.keras

Epoch 1: finished saving model to imdb_sentiment_model.keras
333/333 ━━━━━━━━━━━━━━━━━━━━ 471s 1s/step - accuracy: 0.7143 - auc: 0.7924 - loss: 0.5537 - val_accuracy: 0.8099 - val_auc: 0.8951 - val_loss: 0.4366 - learning_rate: 0.0010
Epoch 2/20
333/333 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.8234 - auc: 0.8878 - loss: 0.4214
Epoch 2: val_auc improved from 0.89514 to 0.91427, saving model to imdb_sentiment_model.keras

Epoch 2: finished saving model to imdb_sentiment_model.keras
333/333 ━━━━━━━━━━━━━━━━━━━━ 459s 1s/step - accuracy: 0.8463 - auc: 0.9078 - loss: 0.3838 - val_accuracy: 0.8392 - val_auc: 0.9143 - val_loss: 0.3855 - learning_rate: 0.0010
Epoch 3/20
333/333 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.8727 - auc: 0.9320 - loss: 0.3297
Epoch 3: val_auc improved from 0.91427 to 0.92314, saving model to imdb_sentiment_model.keras

Epoch 3: finished saving model to imdb_sentiment_model.keras
333/333 ━━━━━━━━━━━━━━━━━━━━ 493s 1s/step - accuracy: 0.8779 - auc: 0.9373 - loss: 0.3170 - val_accuracy: 0.8445 - val_auc: 0.9231 - val_loss: 0.3849 - learning_rate: 0.0010
Epoch 4/20
333/333 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.8753 - auc: 0.9365 - loss: 0.3213
Epoch 4: val_auc did not improve from 0.92314
333/333 ━━━━━━━━━━━━━━━━━━━━ 499s 1s/step - accuracy: 0.8676 - auc: 0.9301 - loss: 0.3373 - val_accuracy: 0.8448 - val_auc: 0.9201 - val_loss: 0.3962 - learning_rate: 0.0010
Epoch 5/20
333/333 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.8999 - auc: 0.9560 - loss: 0.2663
Epoch 5: val_auc improved from 0.92314 to 0.92399, saving model to imdb_sentiment_model.keras

Epoch 5: finished saving model to imdb_sentiment_model.keras
333/333 ━━━━━━━━━━━━━━━━━━━━ 503s 1s/step - accuracy: 0.9086 - auc: 0.9612 - loss: 0.2483 - val_accuracy: 0.8512 - val_auc: 0.9240 - val_loss: 0.3695 - learning_rate: 0.0010
Epoch 6/20
333/333 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.9033 - auc: 0.9583 - loss: 0.2585
Epoch 6: val_auc did not improve from 0.92399
333/333 ━━━━━━━━━━━━━━━━━━━━ 438s 1s/step - accuracy: 0.9117 - auc: 0.9633 - loss: 0.2418 - val_accuracy: 0.8472 - val_auc: 0.9174 - val_loss: 0.3732 - learning_rate: 0.0010
Epoch 7/20
333/333 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.9195 - auc: 0.9702 - loss: 0.2178
Epoch 7: val_auc improved from 0.92399 to 0.92974, saving model to imdb_sentiment_model.keras

Epoch 7: finished saving model to imdb_sentiment_model.keras

Epoch 7: ReduceLROnPlateau reducing learning rate to 0.000500000023748727.
333/333 ━━━━━━━━━━━━━━━━━━━━ 450s 1s/step - accuracy: 0.9288 - auc: 0.9752 - loss: 0.1964 - val_accuracy: 0.8629 - val_auc: 0.9297 - val_loss: 0.3806 - learning_rate: 0.0010
Epoch 8/20
333/333 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.9413 - auc: 0.9804 - loss: 0.1708
Epoch 8: val_auc did not improve from 0.92974
333/333 ━━━━━━━━━━━━━━━━━━━━ 446s 1s/step - accuracy: 0.9474 - auc: 0.9842 - loss: 0.1531 - val_accuracy: 0.8512 - val_auc: 0.9276 - val_loss: 0.4313 - learning_rate: 5.0000e-04
Epoch 8: early stopping
Restoring model weights from the end of the best epoch: 5.

✅ Training complete in 3759.4s (62.7 minutes)
   Best epoch        : 5
   Best val_loss     : 0.3695
   Best val_accuracy : 0.8629
   Best val_auc      : 0.9297
```

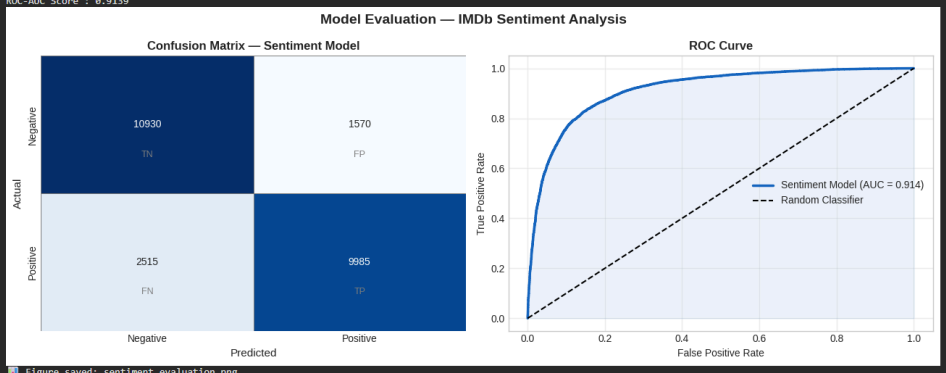
```
"""
```

```
✅ Training complete in 3759.4s (62.7 minutes)
   Best epoch        : 5
   Best val_loss     : 0.3695
   Best val_accuracy : 0.8629
   Best val_auc      : 0.9297
```

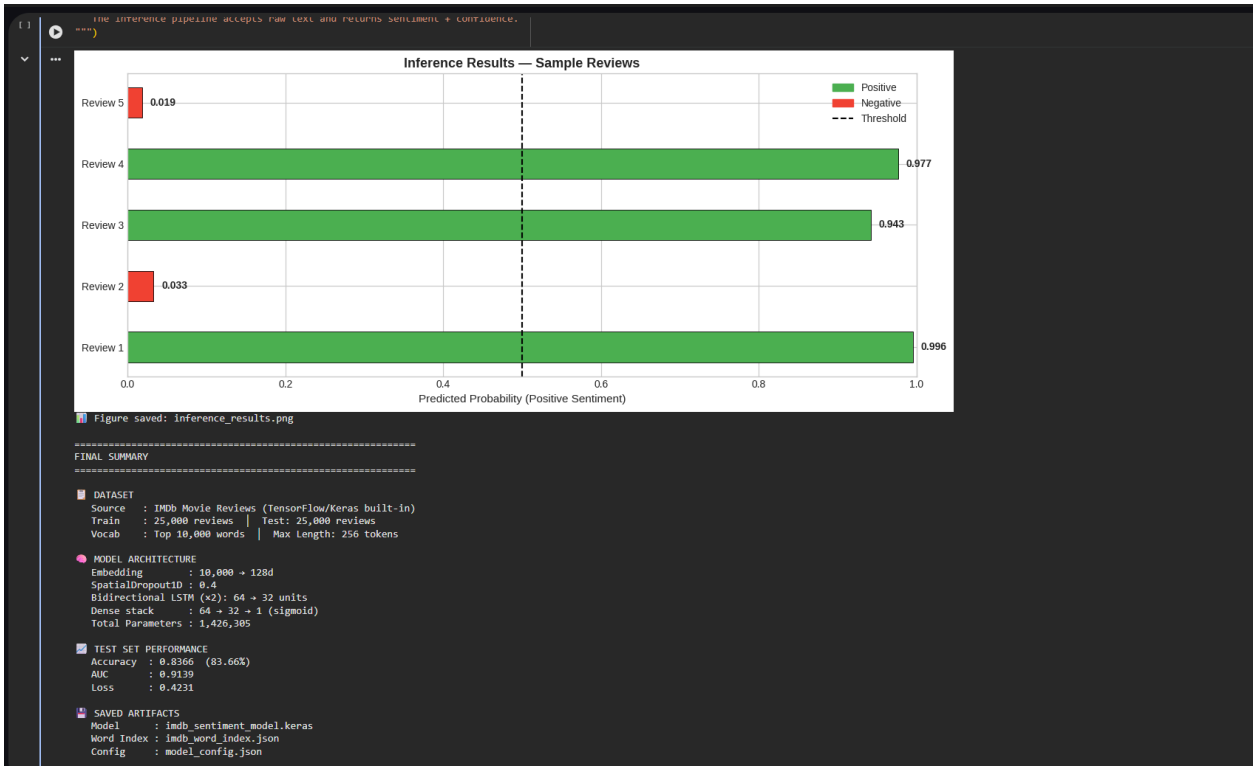

```
📊 Figure saved: training_curves.png
============================================================
EVALUATION ON TEST SET
============================================================
✅ Loaded best model from: imdb_sentiment_model.keras

  Test Loss     : 0.4231
  Test Accuracy : 0.8366 (83.66%)
  Test AUC      : 0.9136

  Predicted Positive (≥0.5): 11,555
  Predicted Negative (<0.5) : 13,445
============================================================
CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

    Negative     0.8129    0.8744    0.8426     12500
    Positive     0.8641    0.7988    0.8302     12500

    accuracy                         0.8366     25000
   macro avg     0.8385    0.8366    0.8364     25000
weighted avg     0.8385    0.8366    0.8364     25000
```

```
"""
```

```
ROC-AUC Score : 0.9139
```




```
📊 Figure saved: sentiment_evaluation.png
============================================================
SAVING MODEL & ARTIFACTS
============================================================
✅ Model saved to : imdb_sentiment_model.keras
✅ Word index saved to : imdb_word_index.json
✅ Config saved to : model_config.json
============================================================
EXAMPLE INFERENCE
============================================================

Review (truncated)                         Sentiment          Confidence
------------------------------------------------------------------------
This movie was absolutely fantastic! The a…  POSITIVE 😊        99.6%
A complete waste of time. The plot made no…  NEGATIVE 😞        96.7%
It was okay, nothing special. Some parts w…  POSITIVE 😊        94.3%
One of the greatest films I have ever seen…  POSITIVE 😊        97.7%
Dreadful film. Poor direction, weak script…  NEGATIVE 😞        98.1%
```



---

15. GitHub Repository

GitHub Repository Link:
https://github.com/jiyaiscoding/Internspark-AI-Internship-Task-2.git

Notebook Link:
InternSpark2 Deep Learning.ipynb